

Periféricos y Dispositivos de Interfaz Humana

Universidad de Granada - Curso 2025/2026

Práctica 6: Reconocimiento de imágenes

Autor: Alejandro López Jiménez

Índice

1. Introducción	2
2. Requisitos	2
2.1. Ejercicio 1	2
2.2. Ejercicio 2	5
2.3. Ejercicio 3	8
2.4. Ejercicio 4	11

1. Introducción

En esta práctica he desarrollado cuatro programas en Python utilizando la librería OpenCV para el reconocimiento de imágenes mediante clasificadores Haar Cascade.

Los clasificadores Haar se pueden descargar del repositorio oficial de OpenCV: <https://github.com/opencv/opencv/tree/master/data/haarcascades>

Los modelos utilizados son:

- `haarcascade_frontalface_alt.xml` (ejercicio 1)
- `haarcascade_frontalface_default.xml` (ejercicios 2 y 4)
- `haarcascade_frontalcatface_extended.xml` (ejercicio 3)
- `haarcascade_fullbody.xml` (ejercicio 4)

Para ejecutar los programas es necesario instalar la librería OpenCV con el comando:

```
pip3 install opencv-python
```

2. Requisitos

Los requisitos que se proponen para superar esta práctica son 4 ejercicios:

1. Hacer reconocimiento de caras de personas en imágenes.
2. Hacer reconocimiento de caras de personas en vídeos.
3. Hacer reconocimiento de caras de gatos en vídeos.
4. Hacer simultáneamente el reconocimiento de cuerpos y caras de personas en vídeos.

2.1. Ejercicio 1

Para hacer este ejercicio he usado el clasificador `haarcascade_frontalface_alt.xml`. El programa recibe la imagen como argumento y detecta los rostros que aparecen en ella, dibujando un rectángulo verde alrededor de cada cara encontrada. De primeras usé `detectMultiScale` con `scaleFactor=1.1` y `minNeighbors=5` y conseguí buenas detecciones sin demasiados falsos positivos, sin embargo en la foto de Einstein hay un cuadro con una cara que no era captada por el programa, cosa que conseguí reduciendo el `scaleFactor` al valor **1.05**.

El código del programa es el siguiente:

```
1 import cv2
2 import sys
3
4 #Ejercicio 1: Deteccion de caras en imagenes
5
6 modelo = cv2.CascadeClassifier('haarcascade_frontalface_alt.xml')
7
8 nombre_imagen = sys.argv[1]
9
10 imagen = cv2.imread(nombre_imagen)
11 gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
12
13 caras = modelo.detectMultiScale(gris, scaleFactor=1.05, minNeighbors=5)
14
15 print(f"Se han detectado {len(caras)} rostro(s)")
```

```
16
17 for (x, y, w, h) in caras:
18     cv2.rectangle(imagen, (x, y), (x + w, y + h), (0, 255, 0), 2)
19
20 cv2.imshow('Ejercicio 1 - Caras en imagen', imagen)
21 cv2.waitKey(0)
22 cv2.destroyAllWindows()
```

Capturas de pantalla

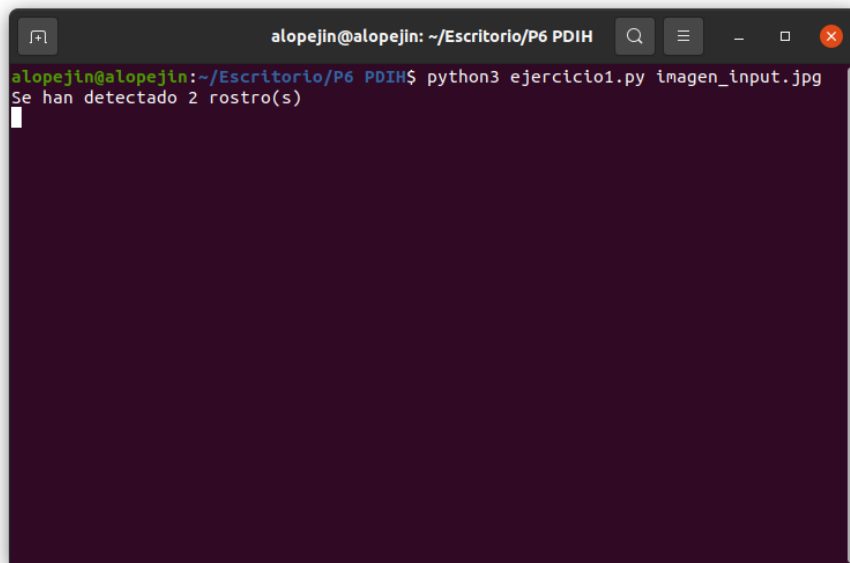


Figura 1: Terminal ejecutando el ejercicio 1

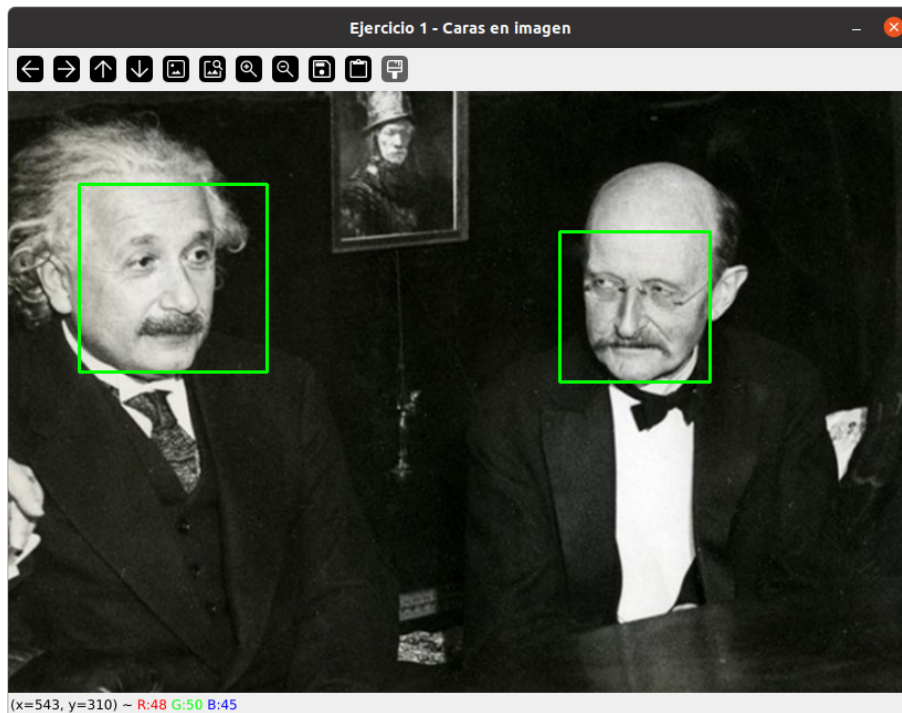


Figura 2: Resultado con scaleFactor=1.1

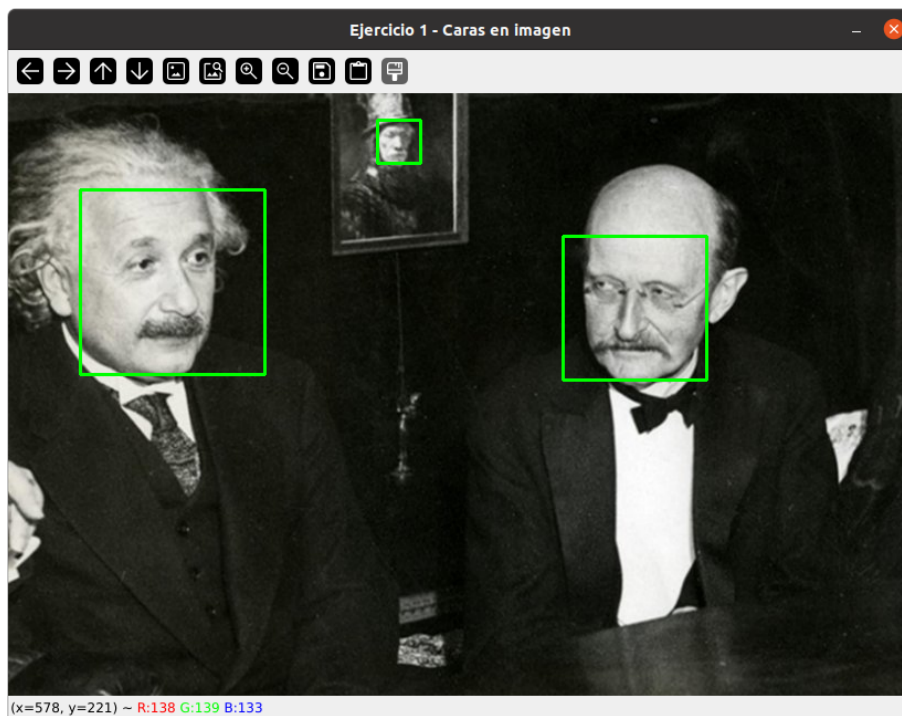


Figura 3: Resultado con scaleFactor=1.05

2.2. Ejercicio 2

En este ejercicio el programa lee un vídeo frame a frame y en cada frame se aplica el clasificador de caras `haarcascade_frontalface_default.xml`. Las caras detectadas se marcan con un rectángulo azul en tiempo real.

El código del programa es el siguiente:

```
1 import cv2
2 import sys
3
4 #Ejercicio 2: Deteccion de caras en videos
5
6 modelo = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
7
8 nombre_video = sys.argv[1]
9
10 video = cv2.VideoCapture(nombre_video)
11
12 while video.isOpened():
13     ret, frame = video.read()
14     if not ret:
15         break
16
17     gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
18
19     caras = modelo.detectMultiScale(gris, scaleFactor=1.1, minNeighbors=4)
20
21     for (x, y, w, h) in caras:
22         cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)
23
24     cv2.imshow('Ejercicio 2 - Caras en video', frame)
25     if cv2.waitKey(1) & 0xFF == 27:
26         break
27
28 video.release()
29 cv2.destroyAllWindows()
```

Capturas de pantalla

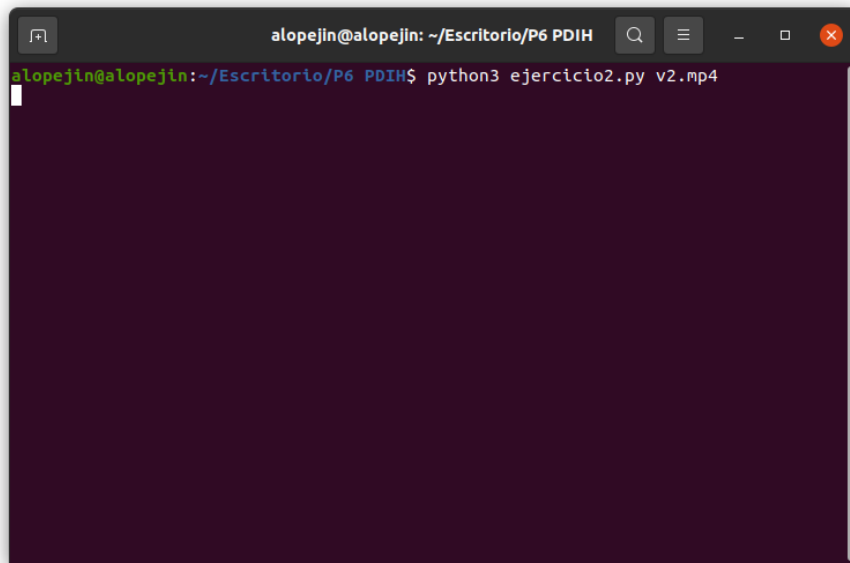


Figura 4: Terminal ejecutando el ejercicio 2

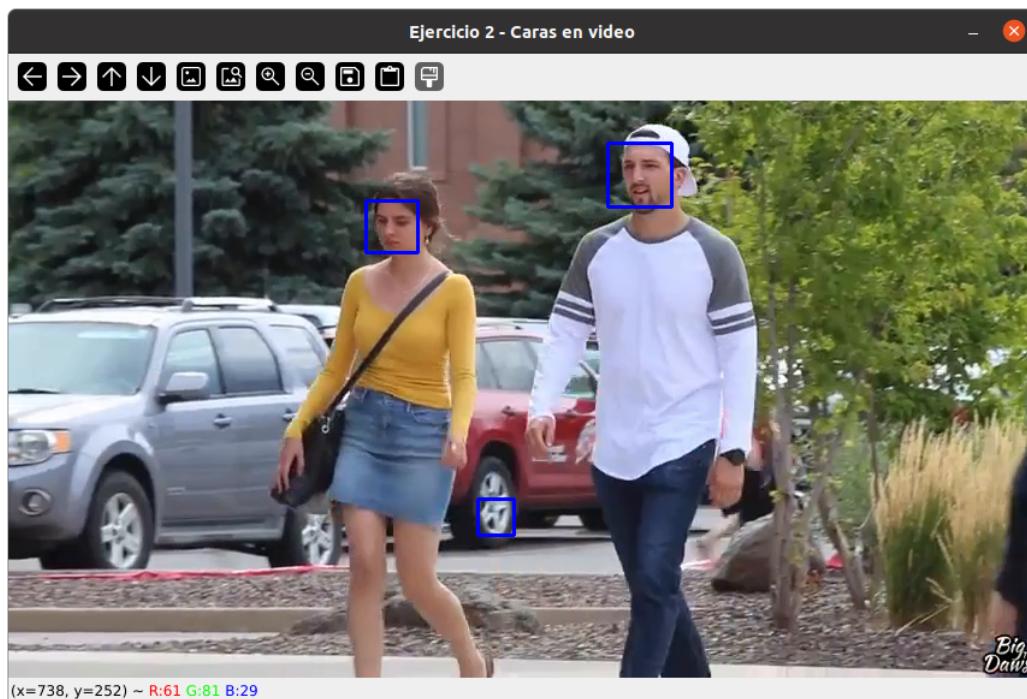


Figura 5: Detección de caras en vídeo (escena 1)

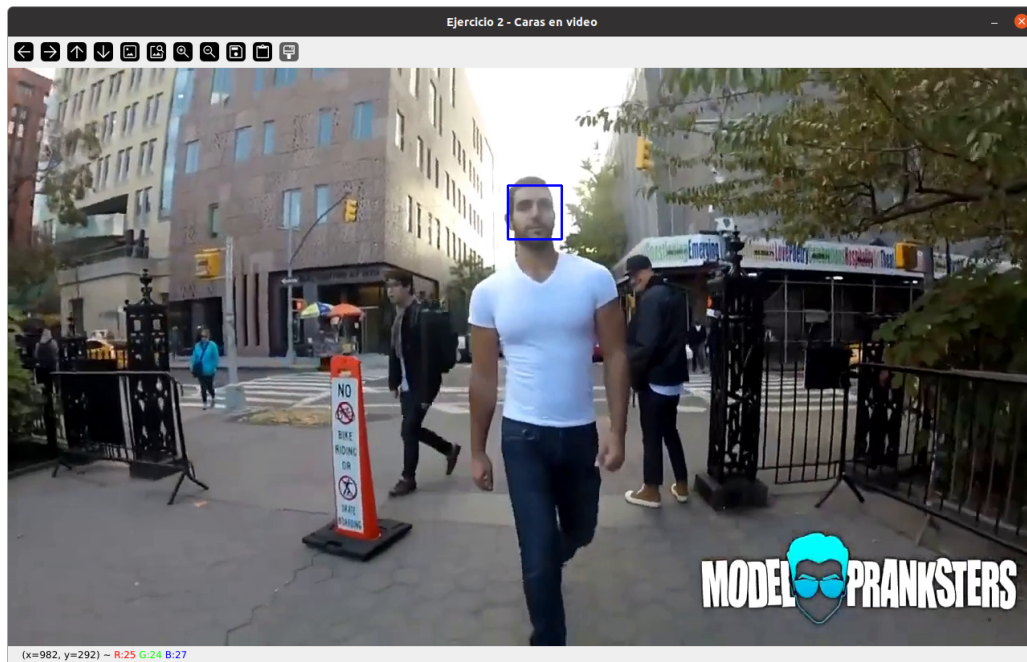


Figura 6: Detección de caras en vídeo (escena 2)

2.3. Ejercicio 3

Este ejercicio es similar al anterior pero utilizando el clasificador entrenado para detectar caras de gatos, `haarcascade_frontalcatface_extended.xml`, con un `scaleFactor=1.05`, más bajo para una mayor precisión, ya que sus caras son más pequeñas más que las de los humanos y más variadas. Cada detección se marca con un rectángulo naranja y la etiqueta “Gato”.

El código del programa es el siguiente:

```
1 import cv2
2 import sys
3
4 #Ejercicio 3: Deteccion de caras de gatos en videos
5
6 modelo = cv2.CascadeClassifier('haarcascade_frontalcatface_extended.xml'
7 )
8 nombre_video = sys.argv[1]
9
10 video = cv2.VideoCapture(nombre_video)
11
12 while video.isOpened():
13     ret, frame = video.read()
14     if not ret:
15         break
16
17     gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
18
19     gatos = modelo.detectMultiScale(gris, scaleFactor=1.05, minNeighbors
20 =5)
21
22     for (x, y, w, h) in gatos:
23         cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 165, 255), 2)
24         cv2.putText(frame, 'Gato', (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX
25 , 0.7, (0, 165, 255), 2)
26
27     cv2.imshow('Ejercicio 3 - Gatos en video', frame)
28     if cv2.waitKey(1) & 0xFF == 27:
29         break
30
31 video.release()
32 cv2.destroyAllWindows()
```

Capturas de pantalla

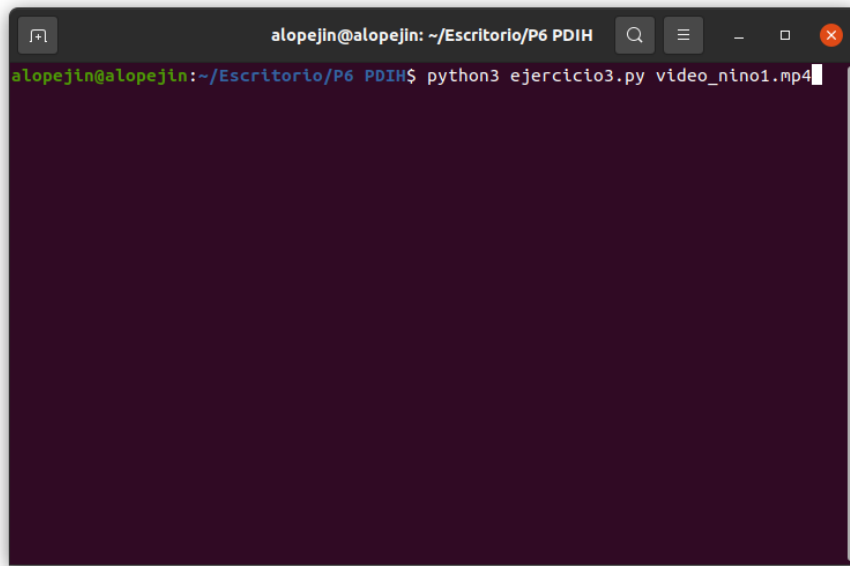


Figura 7: Terminal ejecutando el ejercicio 3

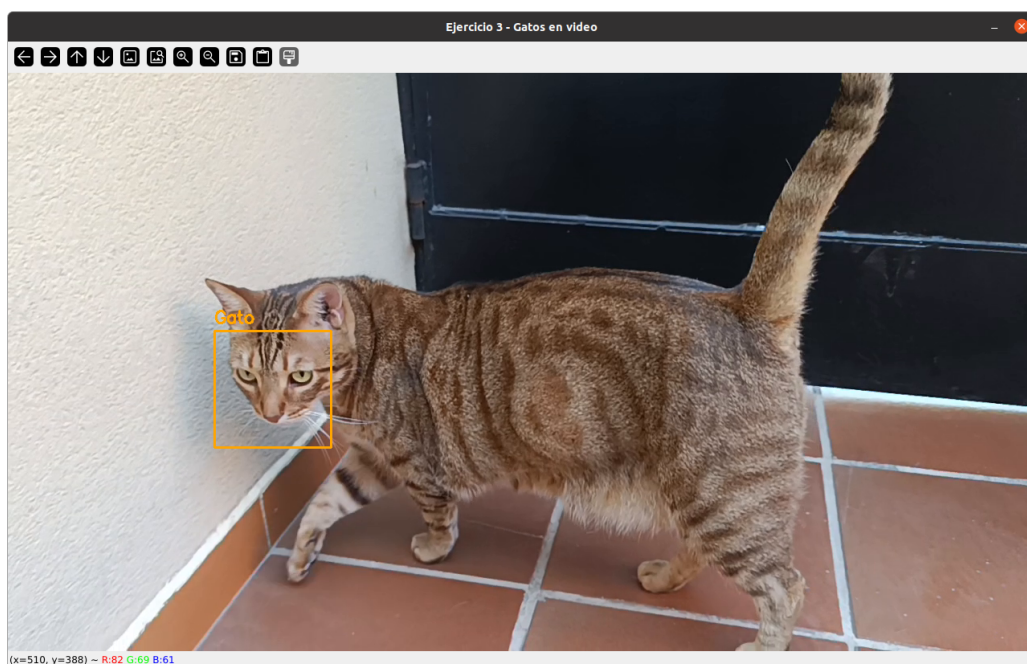


Figura 8: Detección la cara de mi gato de pie

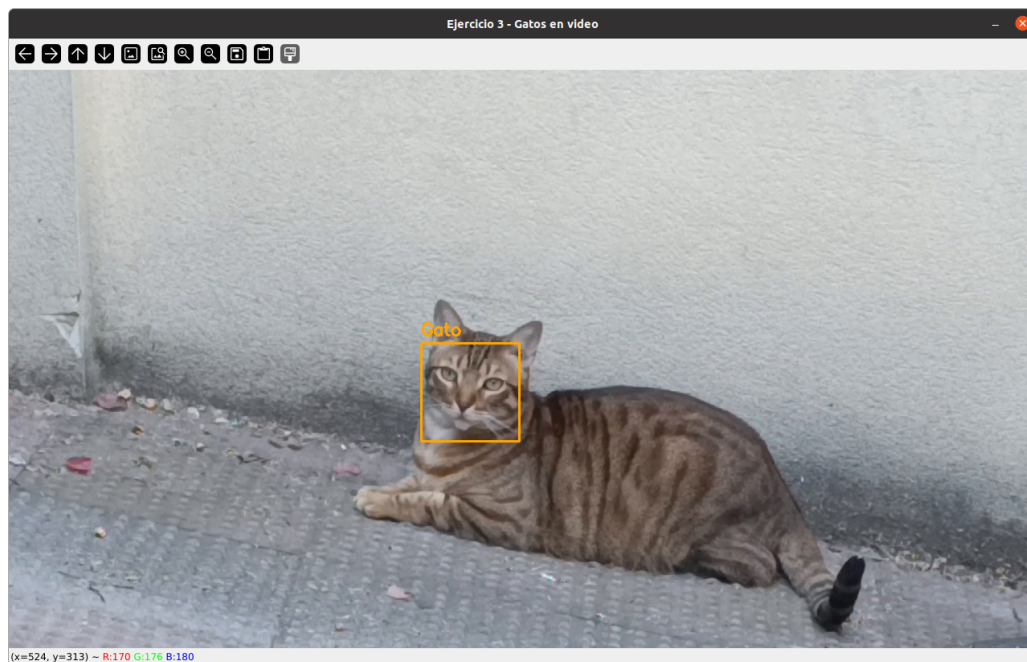


Figura 9: Detección de la cara de mi gato tumbado

2.4. Ejercicio 4

En este ejercicio final hay que detectar tanto caras como cuerpos simultáneamente. Para conseguirlo el programa utiliza los clasificadores `haarcascade_frontalface_default.xml` y `haarcascade_fullbody.xml`. Se cargan simultáneamente los clasificadores y en cada frame se ejecutan ambas detecciones: las caras se marcan en verde con la etiqueta “Cara” y los cuerpos en rojo con la etiqueta “Cuerpo”.

El código del programa es el siguiente:

```
1 import cv2
2 import sys
3
4 #Ejercicio 4: Deteccion simultanea de cuerpos y caras en videos
5
6 modelo_carra = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
7 modelo_cuerpo = cv2.CascadeClassifier('haarcascade_fullbody.xml')
8
9 nombre_video = sys.argv[1]
10
11 video = cv2.VideoCapture(nombre_video)
12
13 while video.isOpened():
14     ret, frame = video.read()
15     if not ret:
16         break
17
18     gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
19
20     caras = modelo_carra.detectMultiScale(gris, scaleFactor=1.1,
21 minNeighbors=4)
22     cuerpos = modelo_cuerpo.detectMultiScale(gris, scaleFactor=1.1,
23 minNeighbors=3)
24
25     for (x, y, w, h) in caras:
26         cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
27         cv2.putText(frame, 'Cara', (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX,
28 0.6, (0, 255, 0), 2)
29
30     for (x, y, w, h) in cuerpos:
31         cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 0, 255), 2)
32         cv2.putText(frame, 'Cuerpo', (x, y - 10), cv2.
33 FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
34
35     cv2.imshow('Ejercicio 4 - Cuerpos y caras', frame)
36     if cv2.waitKey(1) & 0xFF == 27:
37         break
38
39 video.release()
40 cv2.destroyAllWindows()
```

Capturas de pantalla

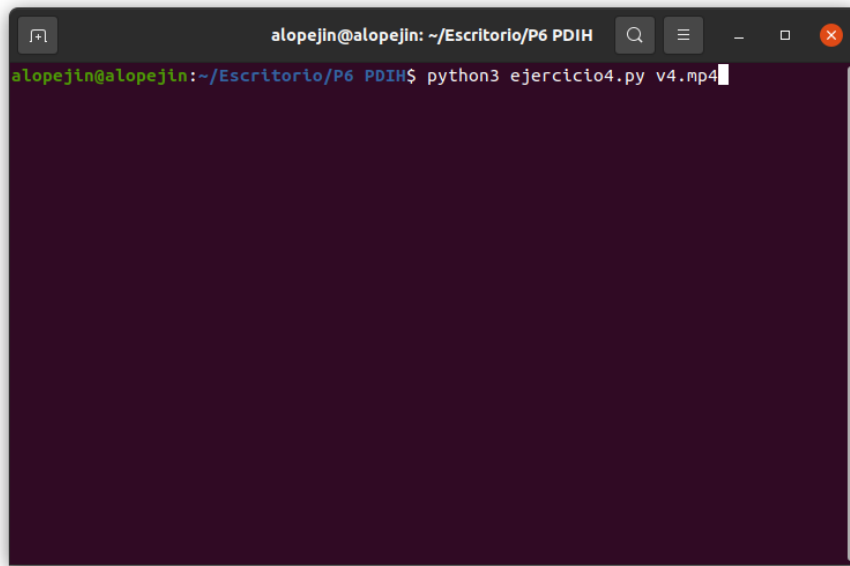


Figura 10: Terminal ejecutando el ejercicio 4

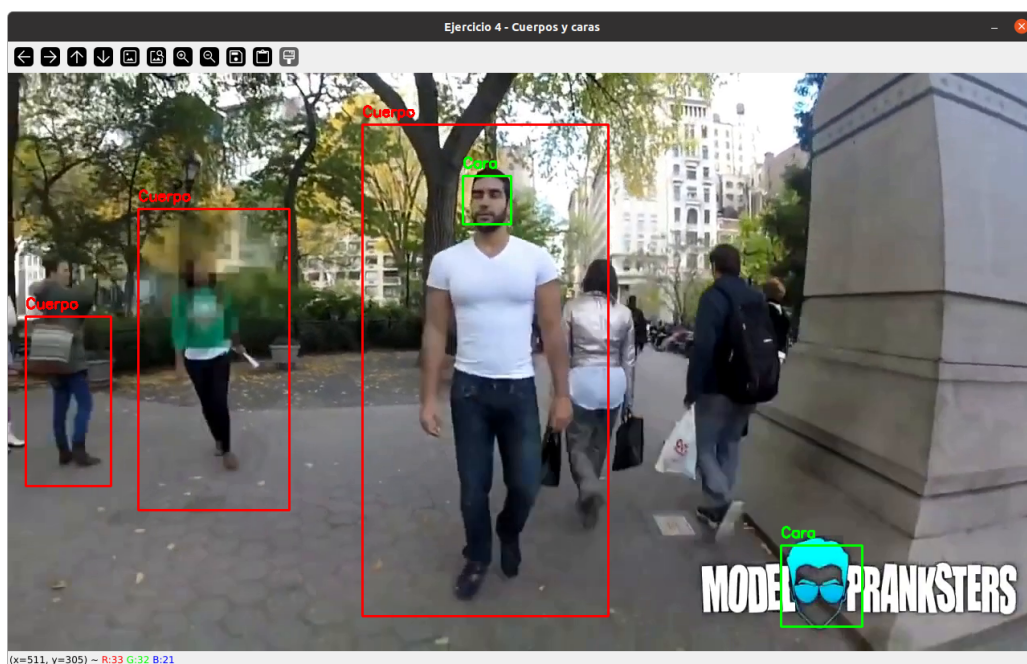


Figura 11: Detección simultánea de cuerpos (rojo) y caras (verde) en vídeo

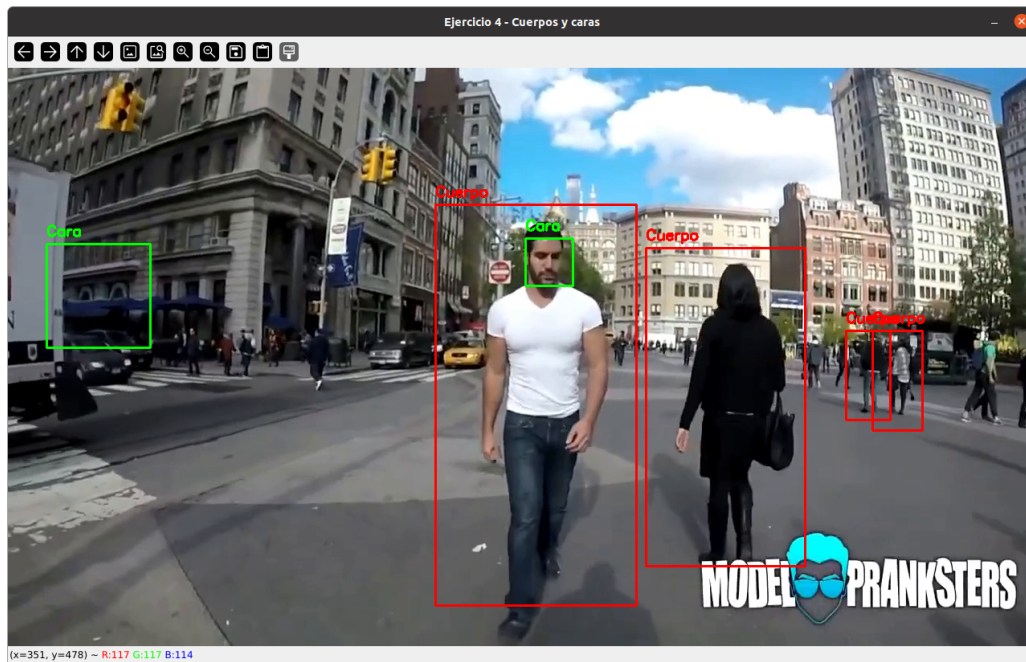


Figura 12: Detección simultánea en otro vídeo

En la carpeta Capturas y vídeos procesos-resultados del repositorio en Github hay más fotos y vídeos de los programas funcionando.